EE1003 Introduction to Computer I

Programming Assignment 2: Knight's Treasure Hunt

Due Date: 2025/11/19 (Wed) 23:59:59

Background

The **Knight's Tour** is a classic mathematical puzzle in chess: on an N×N chessboard, a knight moves according to its unique L-shaped rule (two steps in one direction, one step perpendicular) and attempts to visit every square exactly once. This problem dates back to the Middle Ages and remains a well-studied example in graph theory, algorithm design, and artificial intelligence. You can also try playing this game online by visiting [1].

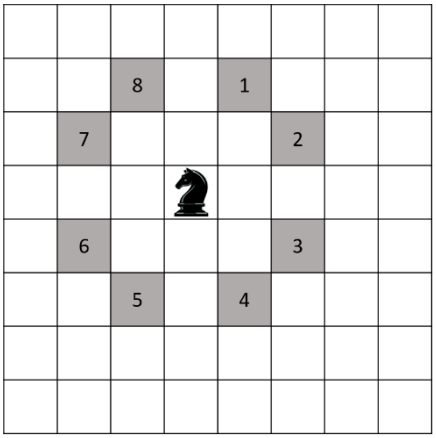


Fig 1: Knight's Tour Board

Depending on whether the knight must return to its starting square, the tour is categorized into two forms:

- **Open Tour:** The knight visits all squares exactly once without needing to return to the start. Equivalent to a Hamiltonian Path on the knight's graph.
- **Closed Tour:** The knight visits all squares exactly once and returns to the starting square. Equivalent to a Hamiltonian Cycle.

From a graph-theoretical perspective, each square can be modeled as a vertex, and each valid knight move as an edge. The Knight's Tour thus becomes a Hamiltonian path/cycle problem on this graph.

In general graphs, the Hamiltonian Path problem is **NP-complete**. However, **Schwenk (1991)** provided a full characterization for rectangular chessboards, proving that almost all boards admit knight tours except for a small set of structural exceptions. Therefore, deciding the existence of a tour on a rectangular board lies in class **P**, not NP-complete.

That said, finding all optimal tours, or solving variants with additional constraints**,** remains computationally challenging. For instance, on the standard 8×8 chessboard:

- The number of open tours is approximately **19,591,828,170,979,904**.
- The number of closed tours is approximately **13,267,364,410,532**.

Clearly, brute force enumeration is infeasible in practice. Common strategies include:

- **Backtracking:** Straightforward but slow.
- **Warnsdorff's Rule:** A heuristic that prioritizes moving to squares with fewer onward moves.
- **Divide & Conquer:** Decomposes the board into smaller regions, connects sub-tours, and is often used for larger boards.

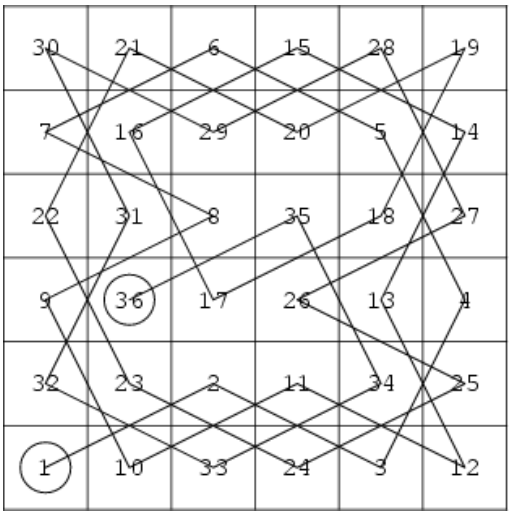


Fig 2: Knight's Tour Path

Description

This assignment adapts the Knight's Tour into a **constrained optimization problem** with items and movement restrictions.

You start from a designated **Start (S)** square. Along the way, you may collect **bonus score (B)**, replenish **energy (E)**, step on **bombs (X)** that reduce your score, or encounter guard **Ray (#)** that cannot be traversed. Additionally, your knight is subject to both **step limits** and **energy constraints**.

Your goal is to find the **highest-scoring path** within the given limits. The solution must be implemented using **recursion (backtracking)** with **pruning**, and the results must be clearly output in text format.

Note: This is an optimization search version, not a full Knight's Tour. **You are not required to visit all squares, and the answer might not be unique. You just need to print the reasonable path.**

Rules

1. Board Tokens (one per square)

- ✓ **S:** Starting point (you start from here).



- ✓ **B:** Score Bonus (adds score).



- ✓ **E:** Energy Complement (restores energy).



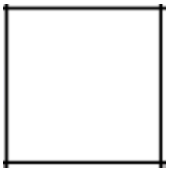
- ✓ **X:** TNT (reduce score).



- ✓ **#:** Guard Ray (impassable).



- ✓ **O:** Empty square (normal step).



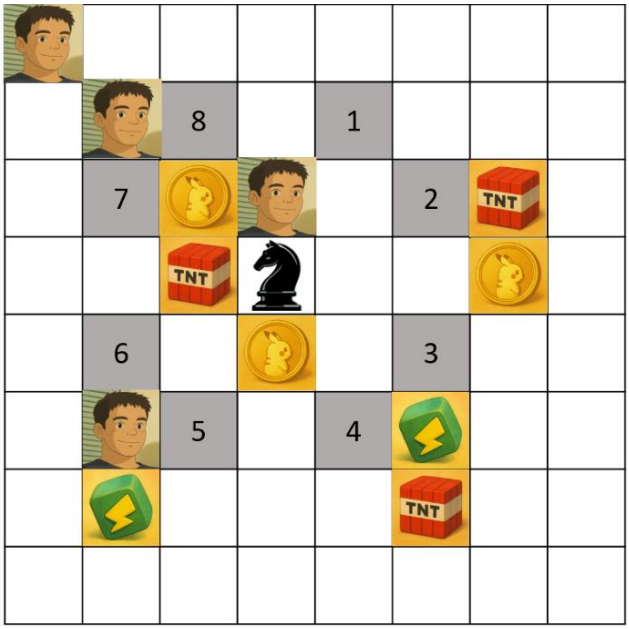


Fig 3: Knight's Treasure Hunt Diagram

2. Movement Rules

- **Each valid move +1 score**.
- Knight moves in an L-shape (standard chess rule).
- Cannot land on #.
- Cannot revisit any previously visited square.
- **Energy constraint:** Start with given energy. Each move consumes 1 energy. If energy ≤0, you cannot move.
- **Step limit:** Total moves cannot exceed the maximum step count.
- **You don't have to use all of your energy. The knight may terminate early if no further safe or beneficial moves exist.**

Input Format

1. First line: **board size N, step limit L, starting energy E**. ($0 < N \leq 20$, $0 < L, E \leq 100$)
   - Example: 7 25 5 (space-separated)
2. Second line: **bonus score S, energy value V, bomb penalty P.** ($0 < S, V, P \leq 10$)
   - Example: 5 4 10 (space-separated)
3. Next N lines: **the board configuration**, each containing N tokens.
   Valid tokens: S B E X # O (space-separated)

- Example (N=4):

```
S O O O
O X X #
O X O O
O O B O
```

Output Format

1. Summary

The program must print the best score and steps, in order:

```
----The Result Of Knight's Treasure Hunt----
Best score: <integer>
Steps: <number of steps>
```

2. Replay

Each line must display **step number**, **coordinates (x,y)** in Cartesian coordinates with origin (0,0) at the bottom-left corner, **event type**, **current score**, and **current energy**.
Example:

```
----The Replay Of Knight's Treasure Hunt----
Step  (x,y)    Type       Score    Energy
-----------------------------------------------
0     (0,6)    start      0        2
1     (2,5)    normal     1        1
2     (4,6)    energy     2        4
```

3. Best Path Grid

Print the board again, but overlay the visited path:

- The starting square remains S.
- Each visited square shows the step index (**starting from 1**).

- Other squares show their **original token**.
- Columns should be properly aligned. **You should set 2 + (maxstep's digits) and use setw for every cell to align columns.**

Example:

```
----The Best Path Grid Of Knight's Treasure Hunt----
S   O   #   5   8   O   14
O   4   O   2   13  6   9
O   1   O   7   X   15  X
O   O   3   12  O   10  17
#   O   O   X   16  X   X
O   O   X   O   11  X   O
X   O   #   O   E   O   O
```

Restrictions

- You **must use recursion** to complete this program. However, the use of loops is permitted for tasks such as **exploring the 8 possible knight moves**, **reading the input**, and **printing the replay/output**.
- **Brute-force solutions are not suggested**, you will not pass the time constraints.
- You may use all standard libraries, but the use of **struct or class** is not allowed.
- Violating any restriction will result in **0 points**.

Sample

Input:

```
4 14 4
2 2 1
S O O O
O X X #
O X O O
O O B O
```

Output: **(Step is unique, but the path is not necessarily unique)**

```
Welcome to Knight's Treasure Hunt!!
Enter board size N, step limit, and starting energy (e.g., 7
25 5): 4 14 4
Enter values for Bonus (B), Energy (E), and Bomb penalty (X)
(e.g., 5 4 10): 2 2 1
Enter the board with 4 lines, each with 4 tokens separated
by spaces:
S O O O
O X X #
O X O O
O O B O

----The Result Of Knight's Treasure Hunt----
Best score: 4
Steps: 3

----The Replay Of Knight's Treasure Hunt----
Step  (x,y)     Type        Score     Energy
-------------------------------------------------
0     (0,3)     start       0         4
1     (2,2)     bomb        0         3
2     (0,1)     normal      1         2
3     (2,0)     bonus       4         1

----The Best Path Grid Of Knight's Treasure Hunt----
S  O  O  O
O  X  1  #
2  X  O  O
O  O  3  O
```

Note: ***Italic*** shows the input from keyboard.

Bonus

This version introduces a special hostile token, **Andy (A)**. Andy occupies a square region that **grows over time**, making certain areas of the board **instantly deadly**.

1. Board new Token (one per square)

✓ **A:** Guard Andy (deadly, you should not pass him…).



2. Andy's Growth

- Guard Andy has **four** possible sizes: 1×1 → 3×3 → 5×5 → 7×7**.**
- **Growth checkpoints:**
  At the start (step 0), Andy occupies only his **seed square** (1×1).
  After step 5 (when the knight finishes its 5th move), Andy expands to a **3×3** region centered at his seed.
  After step 10, Andy expands further to **5×5**.
  After step 15, Andy reaches his maximum size **7×7**.
- Any mention of step 20 or beyond is don't care, Andy is already at maximum size.
- **There can be more than one Andy, and all of them will grow simultaneously, replacing whatever token is on their squares.**

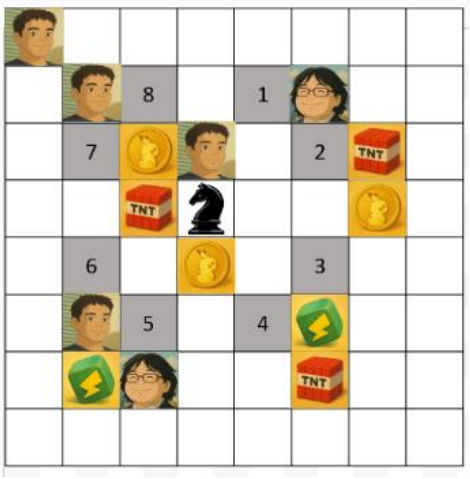


Fig 4: Andy at the start

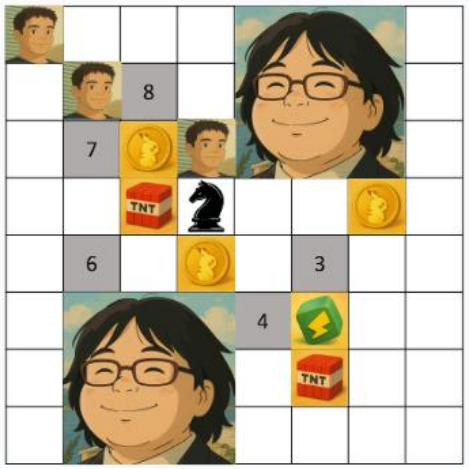


Fig 5: Andy after step 5

3. Death Rule

- **Instant death rule:** After completing a move, and after applying Andy's scheduled growth for that step, **if the knight's square lies within Andy's occupied area, the knight dies immediately.**
- **Validity requirement: However, your program must never output a path in which the knight dies. Even if such a move would yield the highest score,**

if the knight ends that step inside Andy's area, the path is invalid and must be discarded. In other words, you must avoid stepping onto squares that Andy will expand into at growth turns (e.g., steps 5, 10, 15, …).

Output Format

The program must always print a notice when Andy grows immediately, and the notice should be printed **in the same line.**

- After **step 0** (after the start):

```
<!> OH NO <!> ANDY HAS AWAKENED!!! The FINAL COUNTDOWN
begins...
```

- After **step 5** (Andy expands to 3×3):

```
<!> WARNING <!> ANDY IS ENRAGED!!! Run if you can...
```

- After **step 10** (Andy expands to 5×5):

```
<!> DANGER <!> ANDY GROWS STRONGER!!! You feel the end is
near...
```

- After **step 15** (Andy expands to 7×7):

```
<!> CRITICAL <!> ANDY UNLEASHES CHAOS!!! Farewell, hero.
This will be the end of your journey...
```

(If step 20 or later is reached, no new messages are printed.)

If inside Andy's final-area, print A. Andy only **covers original tokens** (O, B, E, X, #) on **unvisited** squares. In other words, if a square was visited, the step number or S **still prints**, even if that square lies inside Andy's area at the final step.

Sample

Input:

```
9 25 5
5 4 3
A O # O E O O O A
O O O O O O E E O
O B O O X O X O O
O O E B O O B # O
# X O X S O X O B
X X X O B X O B O
B X # O E O O O E
O O E B O O B # O
A X O X O X X O A
```

Output: **(Step is unique, but the path is not necessarily unique)**

```
Welcome to Knight's Treasure Hunt!!
Enter board size N, step limit, and starting energy (e.g., 7
25 5): 9 25 5
Enter values for Bonus (B), Energy (E), and Bomb penalty (X)
(e.g., 5 4 10): 5 4 3
Enter the board with 9 lines, each with 9 tokens separated
by spaces:
A O # O E O O O A
O O O O O O E E O
O B O O X O X O O
O O E B O O B # O
# X O X S O X O B
X X X O B X O B O
B X # O E O O O E
O O E B O O B # O
A X O X O X X O A

----The Result Of Knight's Treasure Hunt----
Best score: 57
Steps: 17
```

```
----The Replay Of Knight's Treasure Hunt----
Step  (x,y)    Type        Score     Energy
-------------------------------------------------
0     (4,4)     start       0         5
<!> OH NO <!> ANDY HAS AWAKENED!!! The FINAL COUNTDOWN
begins...
1     (2,5)     energy      1         8
2     (3,7)     normal      2         7
3     (1,6)     bonus       8         6
4     (3,5)     bonus       14        5
5     (5,4)     normal      15        4
<!> WARNING <!> ANDY IS ENRAGED!!! Run if you can...
6     (4,2)     energy      16        7
7     (6,1)     bonus       22        6
8     (8,2)     energy      23        9
9     (6,3)     normal      24        8
10    (8,4)     bonus       30        7
<!> DANGER <!> ANDY GROWS STRONGER!!! You feel the end is
near...
11    (6,5)     bonus       36        6
12    (7,3)     bonus       42        5
13    (5,2)     normal      43        4
14    (3,1)     bonus       49        3
15    (4,3)     bonus       55        2
<!> CRITICAL <!> ANDY UNLEASHES CHAOS!!! Farewell, hero.
This will be the end of your journey...
16    (2,4)     normal      56        1
17    (4,5)     normal      57        0

----The Best Path Grid Of Knight's Treasure Hunt----
A   A   A   A   E   A   A   A   A
A   A   A   2   O   A   A   A   A
A   3   A   A   X   A   A   A   A
A   A   1   4   17  A   11  A   A
#   X   16  X   S   5   X   O   10
A   A   A   A   15  A   9   12  A
A   A   A   A   6   13  A   A   8
A   A   A   14  O   A   7   A   A
A   A   A   A   O   A   A   A   A
```

Note: ***Italic*** shows the input from keyboard.

Submission Requirement

Please submit a source code file (not the entire project) named StudID_PA2.cpp (e.g., 9862534_PA2.cpp) and a report named StudID_Name_PA2_report.pdf (e.g., 9862534_陳聿廣_PA2_report.pdf). If you implement a bonus version, please submit it separately from the original version. Name the source code file of the bonus version StudID_PA2_bonus.cpp (e.g., 9862534_PA2_bonus.cpp) and upload it to ee-class as a separate version from the original. Note that the only acceptable report file format is .pdf, **no .doc/.docx or other files are acceptable. BE SURE to follow the naming rule mentioned above. Otherwise, your program will be not graded.**

We don't restrict the report format and length. In your report, you have to at least describe:

1. How to compile and execute your program; (You can use screenshot to explain)
2. The completion of the assignment; (If you complete all requirements, just specify all)
3. The hardness of this assignment and how you overcome it;
4. Bonus function(s) you implement if any
5. Any suggestions about this programming assignment?

You can also put anything related to the PA in your report, such as pseudocode, control flow diagram, programming developing thought, etc.

Your program will be judged on a workstation with the following command *$PA_studID.out* and with GNU GCC Compiler in C++ 11 standard.

Grading

The grading is as follows:

(1) Correctness of your code: 30% (and 20% is time score)
(2) Readability of your code: 10%
(3) The report: 10%
(4) Demo session: 30%
(5) Bonus: 12% (and 10% is time score)

Even with bonus points, your final score will be capped at **110%**.

Please submit your assignment on time. Otherwise, the penalty rule will apply:

- Within 72hrs delay: 20% off
- More than 3 days: 0 point

Be sure to attend a demo session (the time will be announced later). If you have questions, please E-mail to both me (andyygchen@ee.ncu.edu.tw) and TA 楊 澤 全

(kevin2004812111@gmail.com)

Notices

1. Test case naming

- ✓ Test cases will be named like PA2_PublicCase2_1s.
- ✓ The time limit (e.g., 1s) will be shown in the test case name.
- ✓ Total cases: **4 public** and **6 hidden**.
- ✓ Each case contributes **5%**, with **2%** allocated to the **time constraint**.

2. Bonus test cases

- ✓ There may also be bonus cases, such as PA2_bonus_PublicCase1_1s.
- ✓ Total cases: **4 public** and **6 hidden**.
- ✓ Each case contributes **2%**, with **1%** allocated to the **time constraint**.

3. Execution time measurement

- ✓ You may use the Linux time command to measure runtime.
- ✓ If the **execution time exceeds the specified limit, no time score part will be awarded.**
- ✓ If the program runs longer than 2 minutes without producing output, that test case will receive **0 points**.

4. Hints for learning

- ✓ It is recommended to practice using **pair** and **sort** in C++.
- ✓ Understanding how **warnsdorffs-algorithm** work can be very helpful.

Reference

[1] https://www.maths-resources.com/knights/
[2] https://leetcode.com/problems/minesweeper/description/
[3] https://www.geeksforgeeks.org/dsa/warnsdorffs-algorithm-knights-tour-problem/